BICOL UNIVERSITY - POLANGUI
Web Systems and Technology

ALSOR CO.

BS INFORMATION TECHNOLOGY

PROPONENTS:

Ivan Jacob Milan
Nadel Volante
James Espinosa
Erick John Bonaobra
Manuel Angelo Loma

DearBUP
Feel. Write. Share.

Week 11: Phase 2: Backend Setup

Backend Overview

The Backend in our project follows the MVC or the Model-View-Controller architecture to keep our codes organized and scalable. The models define the data structure for users and posts, while the controllers contain the actual logic for handling requests like login or posting a letter. The routes serves as a road which maps specific URL to corresponding controller function. Middleware provides security layer. Config and Utils folder in opur structure store environment settings and reusable functions to keep it clean and manageable.

1. **Sample Model and Server**
   - **User Model**

```
import mongoose from "mongoose";

const UserSchema = new mongoose.Schema(
  {
    email: { type: String, required: true, unique: true },
    username: { type: String, required: true, unique: true
},
    password: { type: String, required: true },

    // Role for security and middleware authorization
    user_type: {
      type: String,
      enum: ["student", "officer", "department_head",
"admin"],
      required: true,
    },

    display_name: { type: String },

    // Optional improvements (future-ready)
    avatar_url: { type: String, default: "" },
    bio: { type: String, default: "" },

    is_verified: { type: Boolean, default: false },
  },
  { timestamps: true }
);
export default mongoose.model("User", UserSchema);
```

- **Letter Model**

```
import mongoose from "mongoose";

const LetterSchema = new mongoose.Schema({
  user_id: { type: mongoose.Schema.Types.ObjectId, ref:
"User", required: true },
  letter_title: { type: String, required: true },
  letter_content: { type: String, required: true },
  letter_type: { type: String, enum: ["confession",
"gratitude", "concern", "general"] },
  display_name: { type: String },
  is_anonymous: { type: Boolean, default: false },

  // --- NEW SPOTIFY FIELDS ---
  spotifySongName: { type: String },
  spotifyArtist: { type: String },
  spotifyTrackUri: { type: String },
  spotifyImageUrl: { type: String },
  // -------------------------

}, { timestamps: true });

export default mongoose.model("Letter", LetterSchema);
```

- **Post Model**

```
import mongoose from "mongoose";

const CommentSchema = new mongoose.Schema({
  user_id: { type: mongoose.Schema.Types.ObjectId, ref:
"User", required: true },
  text: { type: String, required: true },
  date: { type: Date, default: Date.now }
});

const PostSchema = new mongoose.Schema(
  {
    user_id: { type: mongoose.Schema.Types.ObjectId, ref:
"User", required: true },
    content: { type: String, required: true },

    // New Digital Letter features
    design_template: { type: String, default: "" },
    spotify_track_url: { type: String, default: "" },
```

```
    // New anonymity
    is_anonymous: { type: Boolean, default: false },

    // Social features
    reactions: [{ type: mongoose.Schema.Types.ObjectId, ref:
"User" }],
    comments: [CommentSchema],
    shared_from: { type: mongoose.Schema.Types.ObjectId,
ref: "Post", default: null },

  },
  { timestamps: true }
);

export default mongoose.model("Post", PostSchema);
```

- **Server.js**

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import connectDB from "./config/db.js";
import authRoutes from "./routes/authRoutes.js";
import postRoutes from "./routes/postRoutes.js";
import spotifyRoutes from "./routes/spotifyRoutes.js";


dotenv.config();


connectDB();

const app = express();


app.use(cors());
app.use(express.json());


app.use("/api/auth", authRoutes);
app.use("/api/posts", postRoutes);


app.use("/api/spotify", spotifyRoutes);
```

```
app.get("/", (req, res) => {
  res.send("Backend is running mga lado...");
});


const PORT = process.env.PORT || 5000;

app.listen(PORT, "0.0.0.0", () => {
  console.log(`🚀 Server running on
http://localhost:${PORT}`);
  console.log(`🎵 Spotify Login:
http://127.0.0.1:${PORT}/api/spotify/login`);
});
```

2. **Routes**
   - **Auth Route**

Auth route or Authentication Route serves as the login page. It manages users' access by allowing them to register or login. This route serves as an entry point for security which returns a token that grants the user to access the private features of the web app.

```
import express from "express";
import { register, login } from
"../controllers/authController.js";

const router = express.Router();

router.post("/register", register);
router.post("/login", login);

export default router;
```

- **Post Route**

It handles the main social media features allowing users to create, view, update, and delete letters or posts.Most of these actions are protected by `authMiddleware`, ensuring only logged-in users can interact with content through **reactions, comments, and sharing**.

```
import express from "express";
import {
  createPost,
  getPosts,
  reactToPost,
  addComment,
```

```
  sharePost,
  updatePost,
  deletePost
} from "../controllers/postsController.js";

import { authMiddleware } from
"../middleware/authMiddleware.js";

const router = express.Router();

// Only logged-in users can create, react, comment, update,
delete
router.post("/", authMiddleware, createPost);
router.get("/", getPosts);
router.post("/:id/react", authMiddleware, reactToPost);
router.post("/:id/comment", authMiddleware, addComment);
router.post("/:id/share", authMiddleware, sharePost);
router.put("/:id", authMiddleware, updatePost);
router.delete("/:id", authMiddleware, deletePost);

export default router;
```

- **Spotify Route**

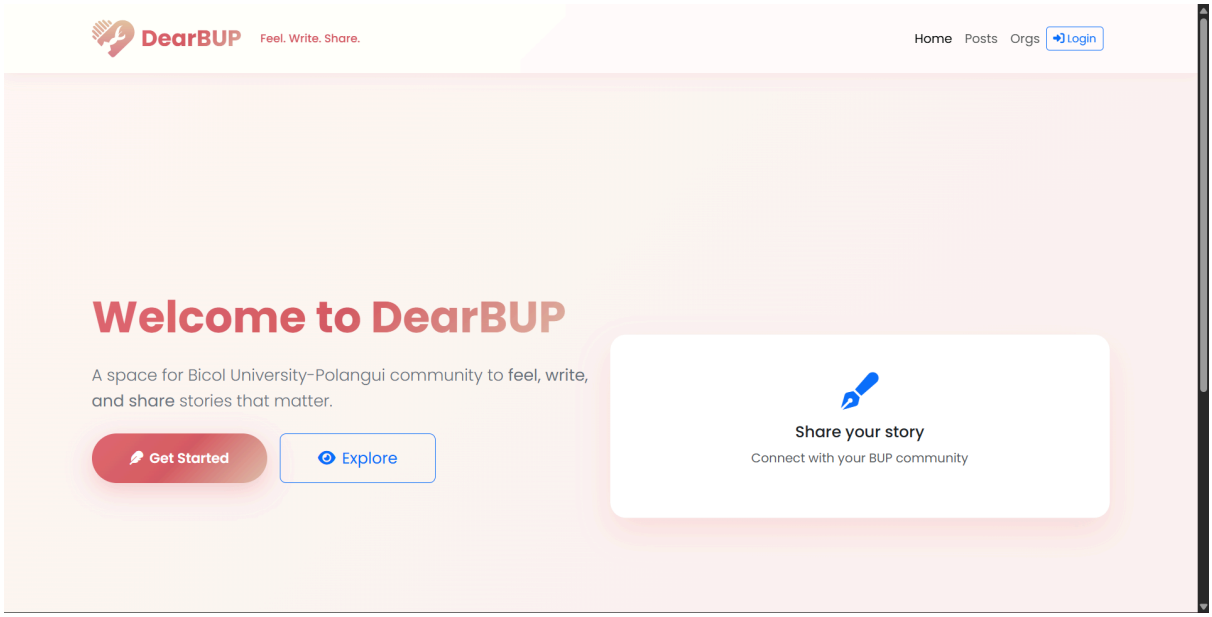
These routes facilitate the OAuth2 flow to connect the website to existing Spotify. It starts with a login and redirect and ends with an access and getting a token via callback. When the user is authenticated, the user can now search for a music track which they can attach to their post.

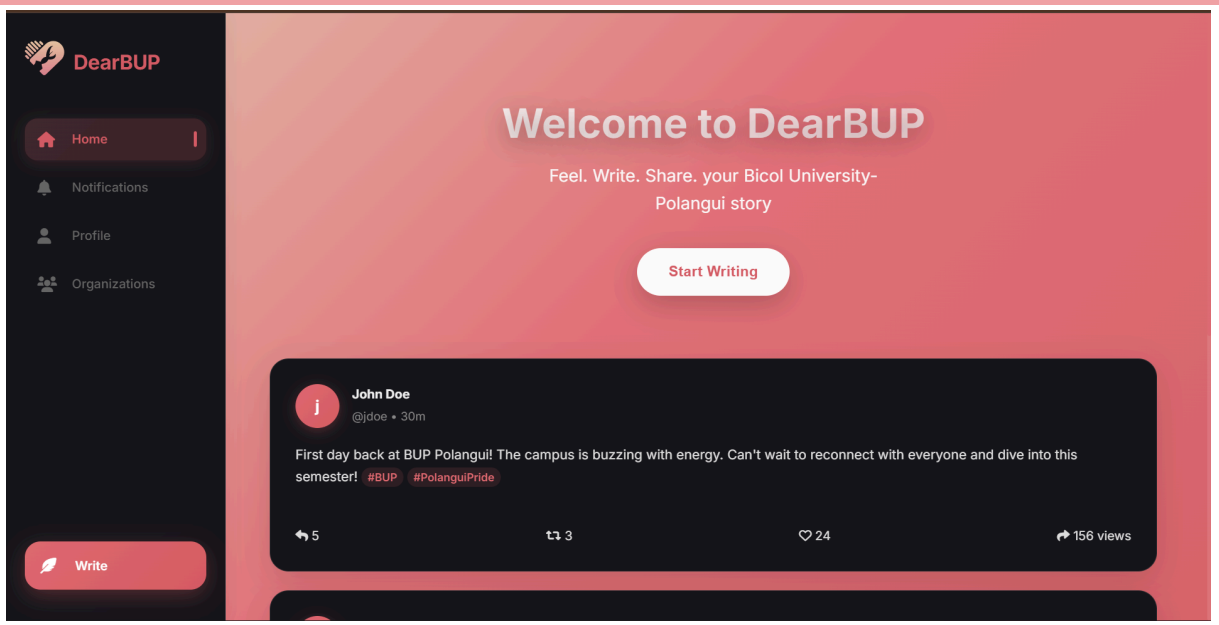
```
import express from "express";
import querystring from "querystring";
import axios from "axios";

const router = express.Router();

// 1. LOGIN ROUTE
router.get("/login", (req, res) => {
  const scope = "user-read-private user-read-email";
  const params = querystring.stringify({
    response_type: "code",
    client_id: process.env.SPOTIFY_CLIENT_ID,
    scope: scope,
    redirect_uri: process.env.SPOTIFY_REDIRECT_URI,
  });
```

```
res.redirect(`https://accounts.spotify.com/authorize?${param
s}`);
});

// 2. CALLBACK ROUTE
router.get("/callback", async (req, res) => {
  const code = req.query.code || null;
  if (!code) return res.status(400).json({ error: "No code
provided" });

  try {
    const response = await axios({
      method: 'post',
      url: 'https://accounts.spotify.com/api/token',
      data: querystring.stringify({
        grant_type: 'authorization_code',
        code: code,
        redirect_uri: process.env.SPOTIFY_REDIRECT_URI,
      }),
      headers: {
        'content-type': 'application/x-www-form-urlencoded',
        'Authorization': 'Basic ' + Buffer.from(
          process.env.SPOTIFY_CLIENT_ID + ':' +
process.env.SPOTIFY_CLIENT_SECRET
        ).toString('base64'),
      },
    });

    const { access_token, refresh_token, expires_in } =
response.data;
    res.json({ message: "Access Token Obtained!",
access_token, refresh_token, expires_in });
  } catch (error) {
    res.status(500).json({ error: "Failed to get token" });
  }
});

// 3.  SEARCH ROUTE
router.get("/search", async (req, res) => {
  const { q, token } = req.query;

  if (!q || !token) {
```

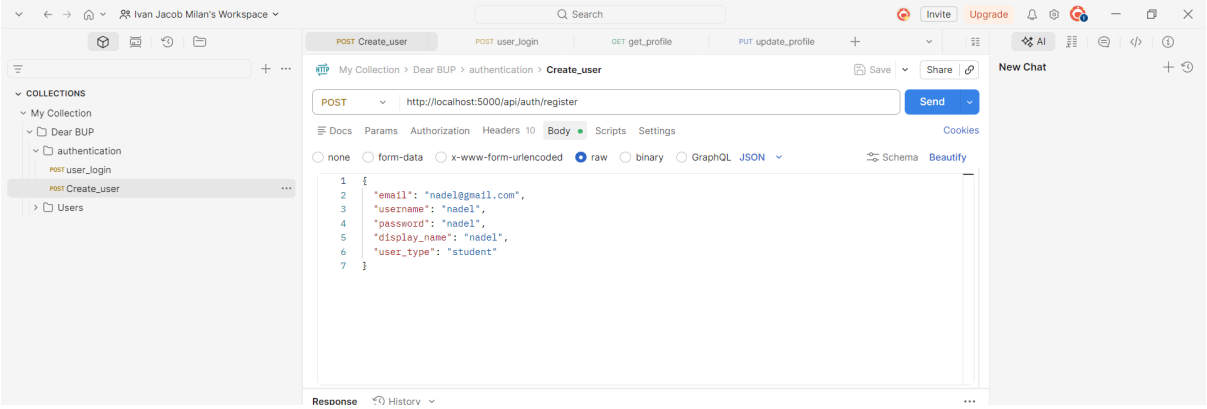
```
    return res.status(400).json({ error: "Missing search
query or token" });
  }

  try {
    const response = await
axios.get("https://api.spotify.com/v1/search", {
      headers: { Authorization: `Bearer ${token}` },
      params: { q: q, type: "track", limit: 10 },
    });

    res.json(response.data.tracks.items);
  } catch (error) {
    console.error("Search Error:", error.response?.data ||
error.message);
    res.status(500).json({ error: "Failed to search for
songs" });
  }
});


export default router;
```

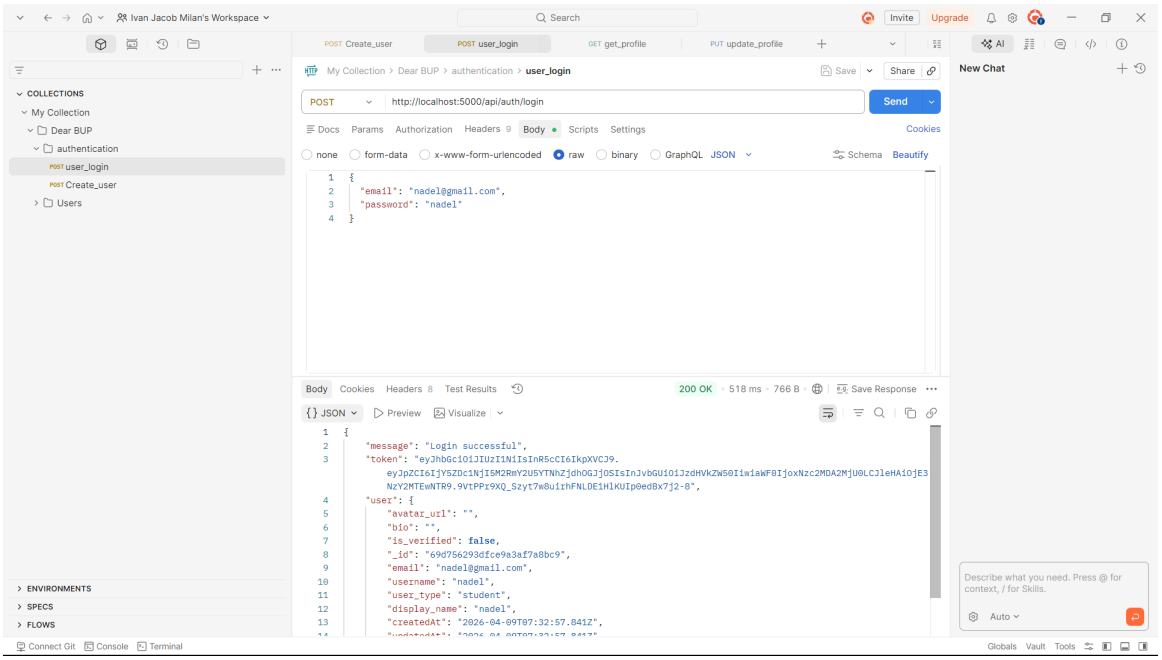
WEB SAMPLE MODEL




DearBUP
Home
Notifications
Profile
Organizations
Write
Welcome to DearBUP
Feel. Write. Share. your Bicol University-Polangui story
Start Writing
John Doe
@jdoe • 30m
First day back at BUP Polangui! The campus is buzzing with energy. Can't wait to reconnect with everyone and dive into this semester! #BUP #PolanguiPride
5
3
24
156 views

API TESTING


Ivan Jacob Milan's Workspace
Search
Invite
Upgrade
POST Create_user
POST user_login
GET get_profile
PUT update_profile
My Collection > Dear BUP > authentication > Create_user
Save
Share
New Chat
COLLECTIONS
My Collection
Dear BUP
authentication
POST user_login
POST Create_user
Users
POST
http://localhost:5000/api/auth/register
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "nadel@gmail.com",
"username": "nadel",
"password": "nadel",
"display_name": "nadel",
"user_type": "student"
}
Response
History


Ivan Jacob Milan's Workspace
Search
Invite
Upgrade
POST Create_user
POST user_login
GET get_profile
PUT update_profile
My Collection > Dear BUP > authentication > user_login
Save
Share
New Chat
COLLECTIONS
My Collection
Dear BUP
authentication
POST user_login
POST Create_user
Users
POST
http://localhost:5000/api/auth/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "nadel@gmail.com",
"password": "nadel"
}
Body
Cookies
Headers 8
Test Results
200 OK
518 ms
766 B
Save Response
JSON
Preview
Visualize
{
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjY5ZDc1NjI5M2RmY2U5YTNhZjdhOGJjOSIsInJvbGUiOiJzdHVkZW50IiwiaWF0IjoxNzc2MDA2MjU0LCJleHAiOjE3
NzY2MTEwNTR9.9VtPPr9XQ_Szyt7w8uizhFNLDE1HlKUIp0edBx7j2-8",
"user": {
"avatar_url": "",
"bio": "",
"is_verified": false,
"_id": "69d756293dfce9a3af7a8bc9",
"email": "nadel@gmail.com",
"username": "nadel",
"user_type": "student",
"display_name": "nadel",
"createdAt": "2026-04-09T07:32:57.841Z",
ENVIRONMENTS
SPECS
FLOWS
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools

REFLECTION

James Espinosa/Documentation Officer

In this week's activity, my role is to explain what was going on with our final project web application in terms of Backend development. I was tasked to document sample models and to explain the role of the routes in our code. As a documentation officer I created an activity tracker to track the progress we've been doing the whole week. The tracker serves as the guide and also for a reminder so that we don't forget the task assigned every week. I am looking forward for the next task and phases for upcoming weeks.

Nadel Volante/Database manager

This week, my primary focus was setting up the MongoDB database connection for our website to ensure the system could properly store and manage data. While my teammates worked on designing the frontend and developing backend features, I configured the database connection using Mongoose and resolved connection issues so the server could communicate successfully with our cloud database. This contribution helped prepare the system's data layer for smooth integration and supported the team's progress as the project continued to develop.

Erick John B. Bonaobra/Frontend Developer

In this week's activity, what I did was the landing page and the main page for our website. I designed the layout and structure to make the interface more user-friendly and visually appealing. Through this, I learned how important UI design is in guiding users through the system. I also gained a better understanding of how the frontend connects with the backend to make the application functional.

Manuel Angelo A. Loma/GitHub Manager

This week, my primary focus as the GitHub Manager was establishing a clean workflow for our development process. After our Backend Manager set up the initial webpage structure and folder hierarchy locally, I coordinated with them to ensure the code was integrated correctly into our shared repository. To prevent merge conflicts and maintain an organized main branch, I instructed the backend lead to develop on a dedicated feature branch. This ensures that all new pushes are reviewed and merged systematically, keeping our group repository stable as the project scales.

Ivan Jacob Milan/ Backend Developer

This week, I first organized the folder structure of the backend to ensure a clean and maintainable project setup. I then focused on building the core structure by creating the main models and setting up essential routes such as authentication, posts, and Spotify integration, along with their corresponding controllers. Lastly, I performed API testing using tools like Postman since the backend is not yet connected to the frontend, allowing me to verify that the endpoints are functioning correctly.